Protocol Audit Report

Version 1.0

Abdullah Amr

June 23, 2026

Prepared by: Abdullah Amr

Lead Auditor:

- Abdullah Amr

Table of Contents

Protocol Summary

The protocol allows only the owner to set and retrieve a password.

Disclaimer

The Abdullah Amr team made every effort to identify as many vulnerabilities as possible within the given time period. However, this report does not guarantee that all vulnerabilities were found.

A security audit is not an endorsement of the underlying business or product. The review was time-boxed and focused only on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
../src/PasswordStore.sol
```

Roles

The protocol defines the following intended roles:

- Owner: Can set and retrieve the password.
- Non-owner: Should not be able to set or retrieve the password.

Executive Summary

The audit was performed using manual review and Foundry-based testing, including fuzz testing where applicable.

The review included:

- Manual inspection of the Solidity codebase
- Foundry unit testing
- Foundry fuzz testing

- Review of access control assumptions
- Review of NatSpec comments and documentation

Issues Found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visible to anyone

Description: All data stored on-chain is publicly visible. Anyone can read blockchain storage directly, regardless of whether a Solidity variable is marked **`private`**.

The `s_password` variable is intended to be private and only accessible through the `getPassword()` function, which is intended to be callable only by the owner. However, the password can still be read directly from contract storage.

Impact: Anyone can read the stored password, severely breaking the intended functionality of the protocol.

Proof of Concept:

Start a local chain:

```
1 anvil
```

Deploy the contract:

```
1 make deploy
```

Example deployed contract address:

```
Contract Address: 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

Read storage slot 1, where `s_password` is stored:

```
cast storage 0x5FbDB2315678afecb367f032d93F642f64180aa3 1 --rpc-url
    http://127.0.0.1:8545
```

Example output:

```
0x6d7950617373776f726400000000000000000000000000000000000000000014
```

Decode the value:

```
cast parse-bytes32-string 0
    x6d7950617373776f726400000000000000000000000000000000000000000014
```

Decoded output:

```
myPassword
```

Recommended Mitigation: Sensitive data such as passwords should not be stored on-chain in plaintext.

The contract architecture should be reconsidered. One possible approach is to encrypt the password off-chain and store only the encrypted value on-chain. However, this still requires careful design because the decryption key must remain off-chain and private.

Additionally, any function that exposes sensitive values should be removed to avoid accidental disclosure.

[H-2] `PasswordStore::setPassword` is callable by anyone

Description: The `PasswordStore::setPassword` function is declared as `external`, but it does not implement access control. According to the function's NatSpec and the intended purpose of the contract, only the owner should be able to set a new password.

```
function setPassword(string memory newPassword) external {
    s_password = newPassword;
    emit SetNetPassword();
}
```

Impact: Anyone can call `setPassword()` and change the stored password.

Proof of Concept: Add the following test to `PasswordStore.t.sol`:

Code

```
function test_anyone_can_call_setPassword(address randomAddress) public {
    vm.assume(randomAddress != owner);

    vm.prank(randomAddress);
    string memory expectedPass = "myNewPassword";
    passwordStore.setPassword(expectedPass);

    vm.prank(owner);
    string memory passReceived = passwordStore.getPassword();

    assertEq(passReceived, expectedPass);
}
```

Recommended Mitigation: Add access control to the `setPassword()` function.

```
if (msg.sender != s_owner) {
    revert PasswordStore__NotOwner();
}
```

Alternatively, this check can be implemented using an `onlyOwner` modifier.

```
modifier onlyOwner() {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    _;
}
```

Then apply it to `setPassword()`:

```
function setPassword(string memory newPassword) external onlyOwner {
    s_password = newPassword;
    emit SetNetPassword();
}
```

Informational

[I-1] `PasswordStore::getPassword` has an incorrect NatSpec comment

Description: The NatSpec comment for `PasswordStore::getPassword` includes a `@param newPassword` tag, but the function does not take any parameters.

```
/*
 * @notice This allows only the owner to retrieve the password.
 * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
    ...
}
```

Impact: The NatSpec documentation is inaccurate and may confuse developers, auditors, or users reading generated documentation.

Recommended Mitigation: Remove the incorrect `@param` line.

```
- * @param newPassword The new password to set.
```